

Statistics and Scaling Gallery

Daniel Ari Friedman

ORCID: 0000-0001-6232-9096

Email: daniel@activeinference.institute

DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Statistics and Scaling Gallery	1
1.1 Overview	1
1.2 Latency histogram	1
1.3 Scaling fit	1
1.4 Confidence intervals	4
1.5 Empirical CDF	4
1.6 Reproducibility and test contract	5
1.7 Cross-references	6

1 Statistics and Scaling Gallery

1.1 Overview

This section is the figure surface of the benchmarks-and-statistics layer: every rendering primitive used by section 17_benchmarks_statistics.md lives in `src/quadmath/viz/vis_stats.py` (`plot_latency_hist`, `plot_scaling_loglog`, `plot_ciBars`, `plot_ecdf`). The primitives are input-agnostic — plain numpy arrays in, artists out; they import nothing from the other `src/` modules and receive their matplotlib axes explicitly, so panels compose inside caller-owned figures and nothing renders at import time. The command-line entry is `quadmath/scripts/stats_gallery.py` — a thin orchestrator that sets a headless backend and a fixed seed (`seed=32`), delegates to the gallery function in `src/quadmath/viz/vis_stats.py` (thin-orchestrator contract of `quadmath/scripts/AGENTS.md`), and prints each written path on its own line: those stdout lines are the `make_all_figures` manifest contract. The gallery writes exactly four PNGs — `stats_gallery_latency.png`, `stats_gallery_scaling.png`, `stats_gallery_ci.png`, `stats_gallery_ecdf.png` — and every random draw comes from the fixed seed, so the whole set is deterministic.

1.2 Latency histogram

`plot_latency_hist(times, ax=None, *, bins=20, title=...)` renders the distribution of per-call wall-clock durations measured by `src/quadmath/stats/benchmarks.py::time_callable` (trials summary in 17_benchmarks_statistics.md) as a histogram with `bins` bins on the given axes. The shape of the distribution carries what the mean alone hides: a tight spike means the harness saw a stable workload, while a heavy right tail is exactly what the `median_s / p95_s` fields of `BenchRow` are there to expose. The gallery panel histograms a deterministic synthetic stand-in for such a sample: 256 draws of a lognormal distribution (mean 0, $\sigma = 0.6$, clipped to $[0.05, 5.0]$ in seconds-scale units) taken from the single fixed-seed `numpy.random.default_rng(32)` generator, so re-renders are byte-identical.

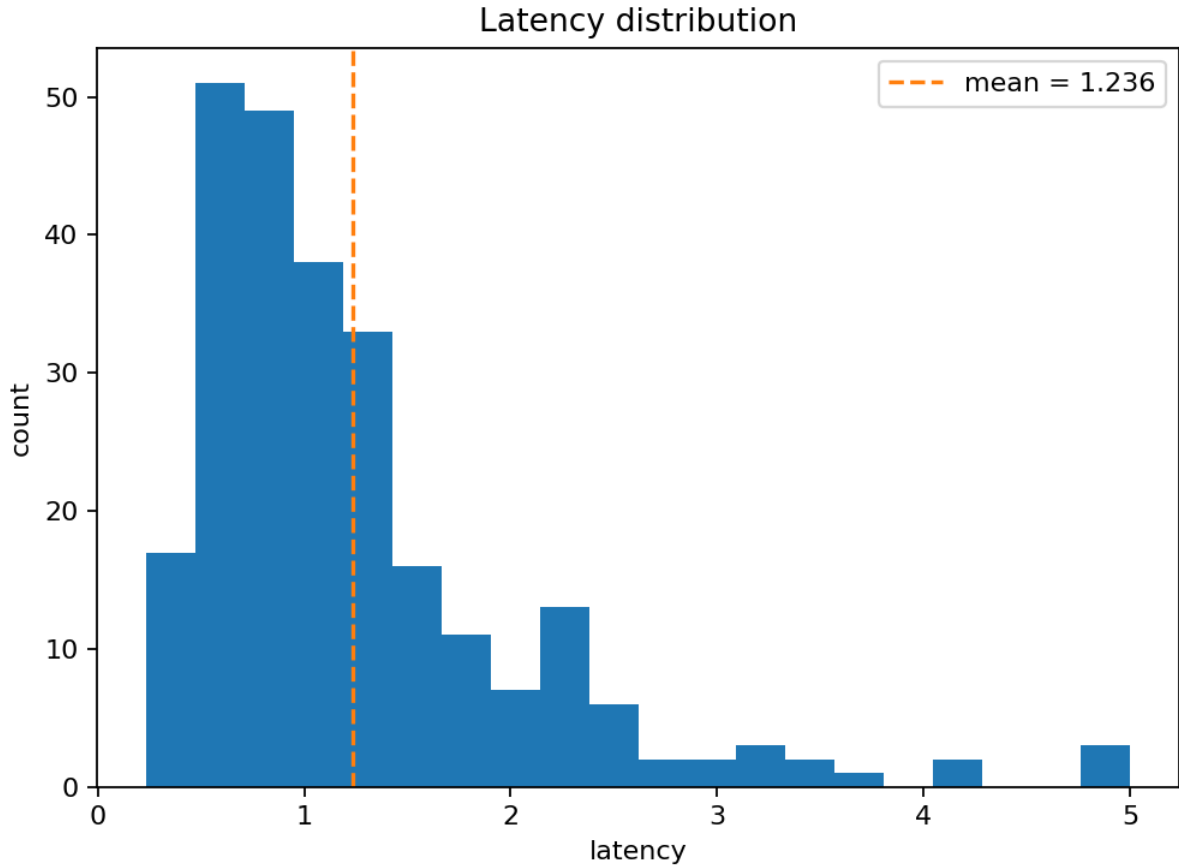


Figure 1: **Latency distribution.** Histogram (20 bins) of a synthetic 256-draw lognormal latency sample (mean 0, $\sigma = 0.6$) clipped to $[0.05, 5.0]$ in seconds-scale units, drawn from the fixed-seed `numpy.random.default_rng(32)` generator of `stats_gallery.py`; the dashed line marks the sample mean. Rendered by `plot_latency_hist`; reproduced by `uv run python quadmath/scripts/stats_gallery.py`. The spread and right tail complement the `mean_s`, `median_s`, and `p95_s` fields of `BenchRow`, whose per-call times `time_callable` measures with `time.perf_counter` after one discarded warmup call.

1.3 Scaling fit

`plot_scaling_loglog(sizes, times, ax=None, *, title=...)` plots measured workload sizes against durations on log-log axes and overlays the least-squares power law of `src/quadmth/stats/statistics.py::scaling_fit` — the slope is the empirical complexity exponent β_1 of (??). On log-log axes a power law is a straight line, so the panel shows at a glance whether a lattice routine scales linearly, quadratically, or worse, and how tightly the data follow the law (r^2). The gallery panel fits a synthetic sweep with known ground truth: durations $t \approx 3 n^{1.35} (1 + \varepsilon)$ with $\varepsilon \sim \mathcal{N}(0, 0.02^2)$ relative noise over sizes $n \in \{8, 16, 32, 64, 128, 256\}$, drawn from the same fixed-seed generator (times rounded to four decimals for byte stability), so the fit should recover the planted exponent 1.35.

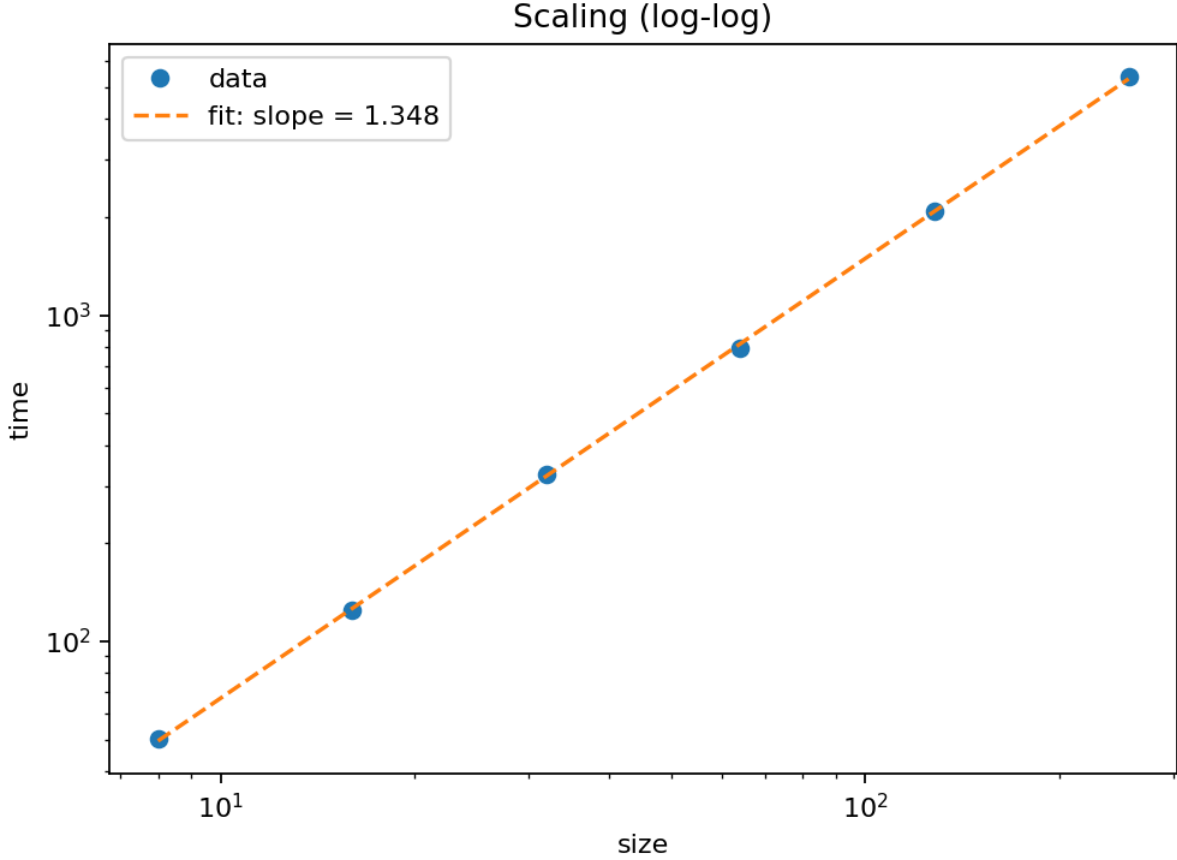


Figure 2: **Log-log scaling fit.** Workload sizes $n \in \{8, 16, 32, 64, 128, 256\}$ (dimensionless units) against synthetic durations $t \approx 3 n^{1.35} (1 + \varepsilon)$ with $\varepsilon \sim \mathcal{N}(0, 0.02^2)$ relative noise, times rounded to four decimals, all drawn from the fixed-seed `numpy.random.default_rng(32)` generator; the least-squares power law from `scaling_fit` is overlaid by `plot_scaling_loglog`, and its fitted slope — annotated in the legend — should recover the planted exponent 1.35, the empirical complexity exponent β_1 of (??).

1.4 Confidence intervals

`plot_ci_bars(labels, means, lows, highs, ax=None, *, title=...)` renders paired estimates as a bar-and-whisker chart: one bar per label at its point estimate, with error bars spanning the low and high ends of a percentile bootstrap interval `src/quadmth/stats/statistics.py::bootstrap_ci` (??). Overlapping intervals visually encode the same information a permutation test quantifies (??): two conditions whose intervals do not overlap are the ones the pooled test flags. The gallery panel shows three benchmark estimates (`to_xyz`, `quadray_from_xyz`, `shell_enum`) with symmetric intervals whose half-widths are seeded uniform draws on $[0.05, 0.25]$ from the same fixed-seed

`numpy.random.default_rng(32)` generator: the primitive renders whatever lows / highs it is given, and in a measured analysis those bounds are the percentile endpoints `bootstrap_ci` returns ((??)).

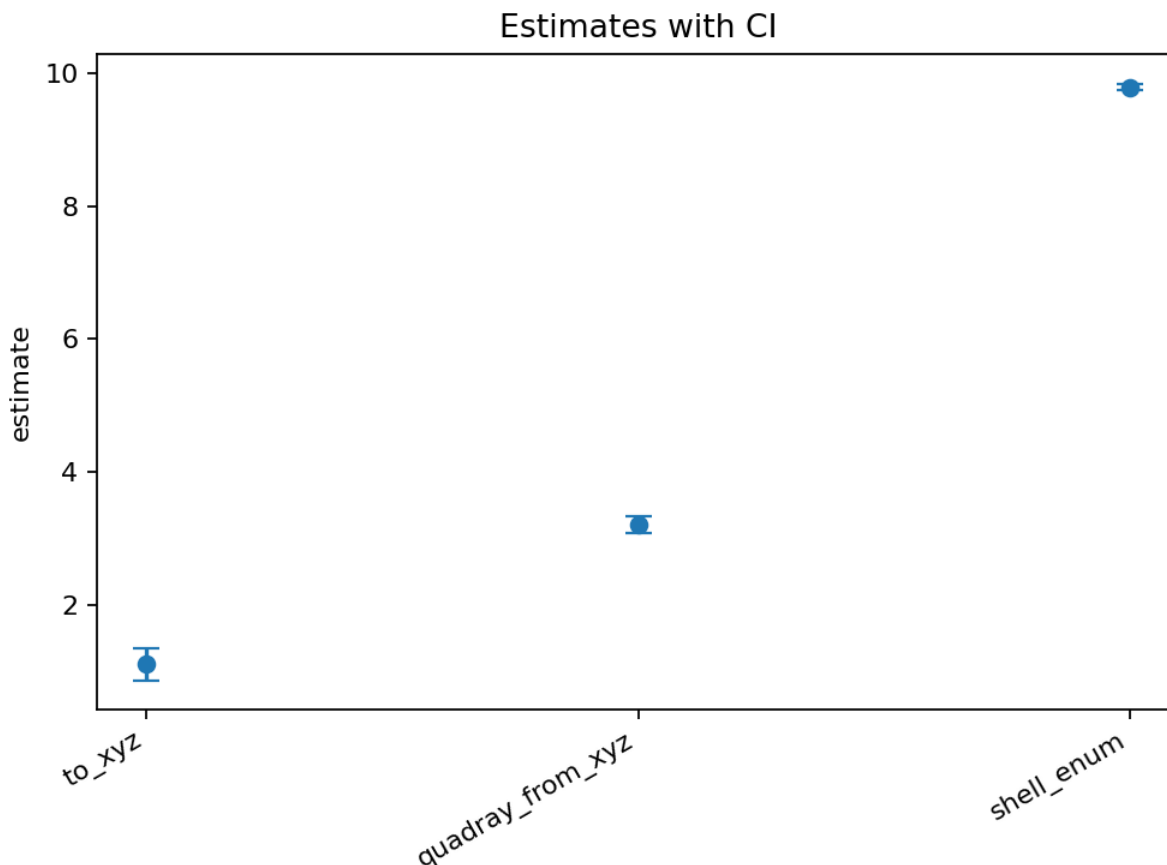


Figure 3: **Point estimates with confidence intervals by condition.** Three benchmark estimates (`to_xyz`, `quadray_from_xyz`, `shell_enum`) in synthetic estimate units, rendered by `plot_ci_bars` as points with symmetric error bars whose half-widths are uniform draws on $[0.05, 0.25]$ from the fixed-seed `numpy.random.default_rng(32)` generator; in a measured analysis the bar ends would be the 95% percentile bootstrap endpoints from `bootstrap_ci` (2000 resamples, (??)), whose non-overlap is the visual cue the permutation test of (??) quantifies.

1.5 Empirical CDF

`plot_ecdf(values, ax=None, *, title=...)` renders the empirical distribution function

$$\hat{F}(t) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{t_i \leq t\}, \quad (1)$$

a monotone step function that jumps by $1/n$ at every observed value — the exact distribution of the sample, with no binning choices at all. For timing data the ECDF answers directly what a percentile summary rounds off: the curve passes through the empirical median at height 0.5 and through the 95th percentile at height 0.95, so `median_s` and `p95_s` are readable off the plot. The gallery panel draws the ECDF of the same 256-draw clipped lognormal latency sample as the histogram panel above, so the two panels read as one distribution in two renderings.

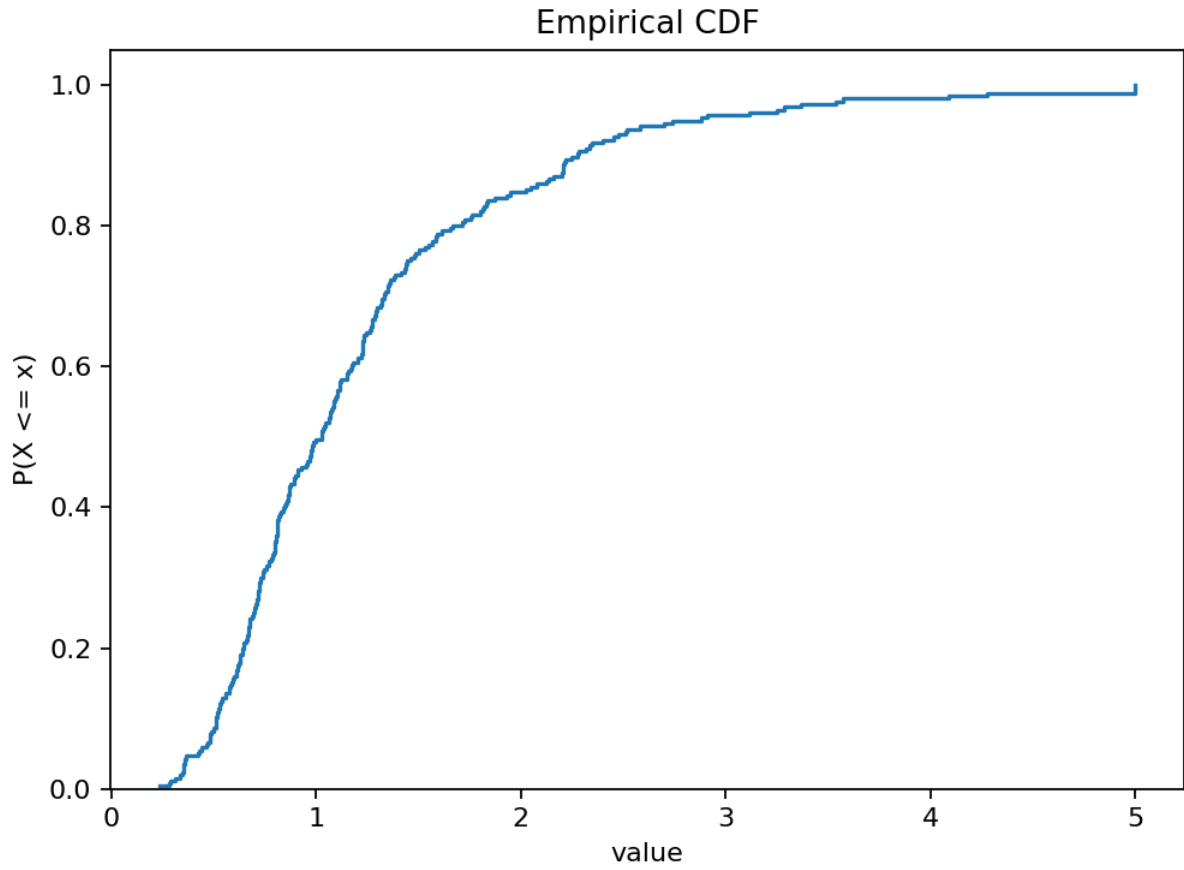


Figure 4: **Empirical cumulative distribution of the latency sample.** Exact step-function ECDF $\hat{F}(t)$ of the same 256-draw clipped lognormal sample as the histogram panel above (fixed seed 32, latency in seconds-scale units), rendered by `plot_ecdf`; the empirical median (height 0.5) and 95th percentile (height 0.95) are read directly off the curve.

1.6 Reproducibility and test contract

- The gallery function in `src/quadmath/viz/vis_stats.py` writes exactly the four files listed above, in that order, into the output directory (creating it when missing) and returns their paths. Every panel is fully seeded — the script fixes `seed=32` — and the PNGs carry no timestamp metadata, so re-renders are byte-identical.
- Tests: `tests/test_vis_stats.py` pins the contract with deterministic assertions, no mocks, and no pixel diffs — artist placement on caller-provided axes, input-agnostic behavior over plain arrays, and the invariants of each primitive (monotone non-decreasing ECDF steps, CI whisker endpoints matching the lows/highs inputs). Consistent with `17_benchmarks_statistics.md`, no test asserts wall-clock timing values.
- Figure regeneration: `uv run python quadmath/scripts/stats_gallery.py` (the script sets `MPLBACKEND=Agg` itself); `std-out` is exactly the four written paths, the same manifest contract as `quadmath/scripts/lattice_gallery.py`.
- Markdown: `uv run python quadmath/scripts/validate_markdown.py`.

1.7 Cross-references

- The measured quantities behind every panel: `17_benchmarks_statistics.md` (`src/quadmath/stats/benchmarks.py`, `src/quadmath/stats/statistics.py`).
- The sibling gallery of the lattice layer, same thin-orchestrator and determinism contract: `16_lattice_gallery.md` (`src/quadmath/viz/vis_lattice.py`).